Service start-up report

Target: **ids (local)**
Recording started (t = 0): 2026-10-17 16:30:25 CEST (UTC+0200)
Report generated: 2026-10-17 16:31:23 CEST
Operation: **start** · Recorded window: 58s · compose exit code: **0**

No warnings: no container failed or was left unstarted, and no service logged a line with an error word during the start.

A tile's red badge counts every log line with an error word ("error", "fail…", "fault…", whatever its level) since the dashboard's log tail last started, including the 50 lines of scrollback it replays, and it keeps counting after the report ends; the Warnings here count only the lines logged during the recording. So the two numbers can differ, and a line logged after a screenshot shows up in a later one.

Resource usage recorded while the stack was brought up. Each dashed red vertical line marks an event (a container created, started, going healthy or unhealthy, or a one-shot job finishing or being removed), numbered in a band above the graph. Container states come from *docker compose ps*, checked every ~1.5s, so event times are good to about a second. Events up to 1.5s after the first event of a number share it (in practice, the changes seen in one status check); the event key under the host graph lists what each number covers, with its time to a tenth of a second. The largest host CPU peak is called out with where it falls among the markers. **Figure 1** lists the containers of this stack, grouped as in the per-container graphs of section 2.

Host CPU and RAM were read from /proc every 0.25s, per-container CPU and memory were read from the containers' cgroup files every 0.25 s. A line is broken wherever no sample was taken for longer than a few sampling intervals, rather than drawn straight across, and a × marks where a container's line ends before the window does (it stopped, was removed, or a one-shot job exited). A container's line starts at its first sample, once it is really running.

No container changed state in the first 5s: docker compose was building or checking the locally built images first (*up --build*).

Without a healthcheck, backup_service, honeypot_content_sync, log_capper, pool_manager, reverse_proxy, vector_outbound never get a “healthy” event: “started” is their last state, so a late “started” marker is that container being ready, not a missing event.

Reading the metrics. **SD** (standard deviation) is the typical distance of a value from the mean. The **95th percentile** is the level usage stayed at or below 95% of the time; a **peak** far above it is a short burst rather than sustained load. **P(≥x)** is the share of the recording spent at or above *x*: the chance that a randomly picked moment was that busy. “How unusual the peak is” counts SDs above the mean; more than about 3 is a rare spike.

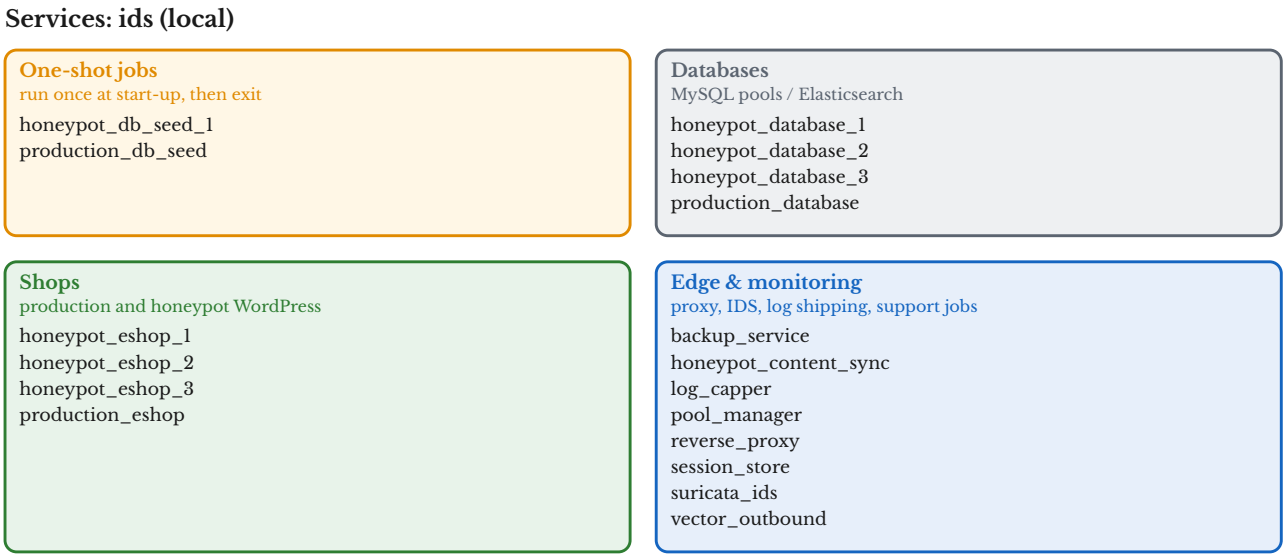
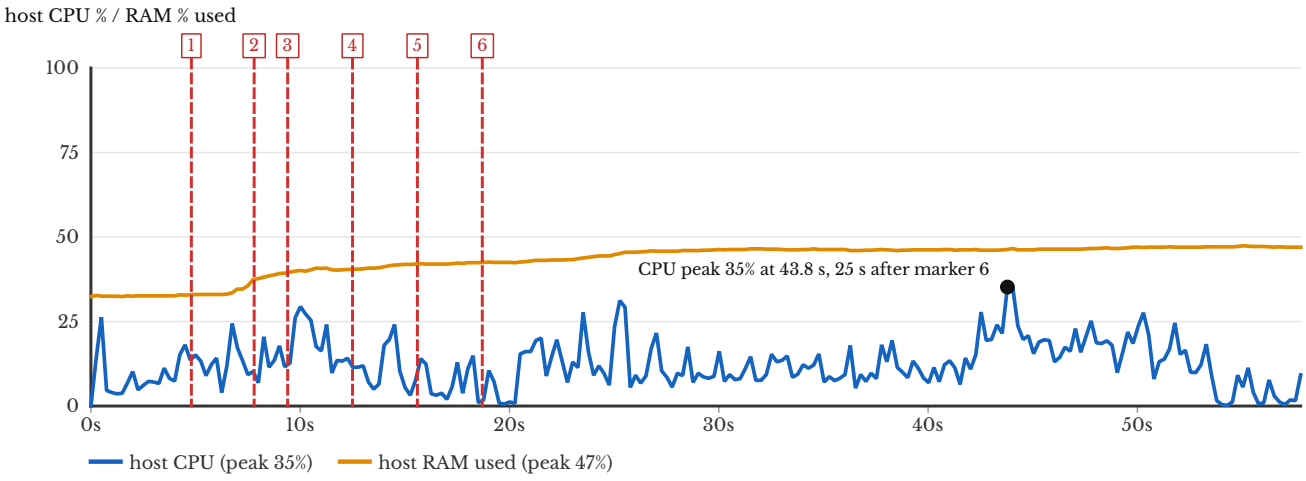


Figure 1. Containers of ids (local) grouped as in the per-container charts in section 2: each box lists exactly the containers those charts plot. The SIEM (Elasticsearch, Kibana) is a separate stack with its own report; here it only receives logs from vector_outbound.

1. Overall system (host)



Event key (marker numbers on all graphs)

Marker	Elapsed	What happened
1	4.8 s–6.3 s	created: backup_service, honeypot_content_sync, honeypot_database_1, honeypot_database_2, honeypot_database_3, honeypot_db_migration, honeypot_db_seed_1, honeypot_db_seed_2, honeypot_db_seed_3, honeypot_eshop_1, honeypot_eshop_2, honeypot_eshop_3, init_setup, pool_manager, production_database, production_db_seed, production_eshop, reverse_proxy, session_store, suricata_ids, vector_outbound, log_capper; finished: init_setup; running, health pending: production_database, suricata_ids; started: log_capper
2	7.8 s	healthy: honeypot_database_1, honeypot_database_3, production_database, session_store; running, health pending: honeypot_database_2; started: production_db_seed
3	9.4 s–10.9 s	healthy: honeypot_database_2, suricata_ids; finished: honeypot_db_migration, production_db_seed; started: honeypot_db_seed_1, pool_manager, honeypot_db_seed_2, honeypot_db_seed_3; running, health pending: production_eshop
4	12.5 s–14.0 s	started: honeypot_content_sync, backup_service; finished: honeypot_db_seed_1, honeypot_db_seed_2, honeypot_db_seed_3; running, health pending: honeypot_eshop_1, honeypot_eshop_2, honeypot_eshop_3; healthy: production_eshop
5	15.6 s–17.1 s	healthy: honeypot_eshop_1, honeypot_eshop_2, honeypot_eshop_3; started: reverse_proxy
6	18.7 s	started: vector_outbound

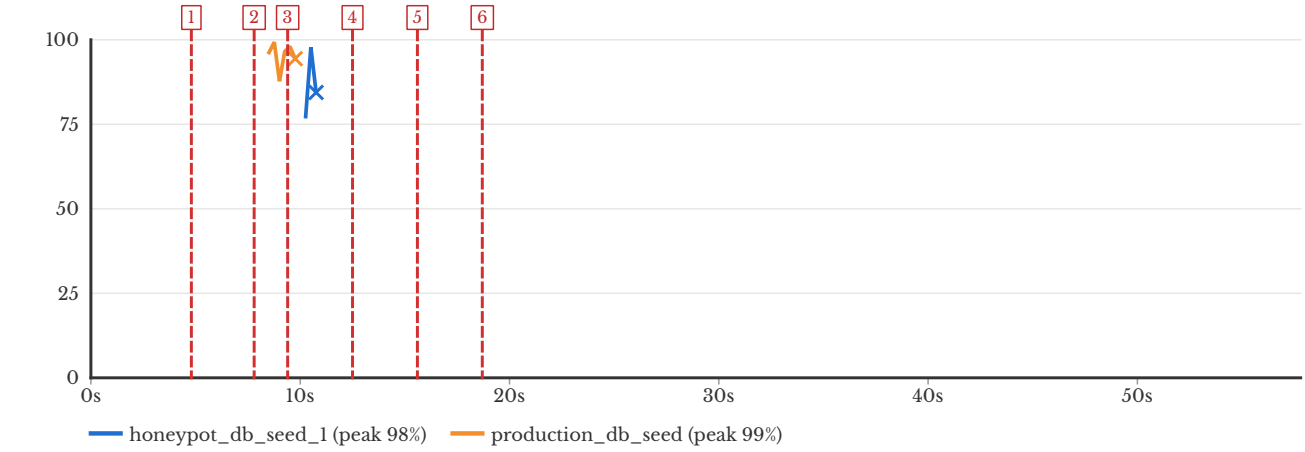
Probability metrics: host		
Host CPU, mean ± SD	**12.3% ± 7.1**	average over 232 samples; the SD is the typical swing around it
Host CPU, 95th percentile / peak	**25.3% / 35.2%**	95% of the time usage was at or below the first figure
Host CPU, how unusual the peak is	**3.2 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host CPU, P(≥50%)	**0.0%**	chance a random moment of this recording was at least this busy
Host CPU, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, mean ± SD	**43.2% ± 4.7**	average over 232 samples; the SD is the typical swing around it
Host RAM used, 95th percentile / peak	**47.1% / 47.4%**	95% of the time usage was at or below the first figure
Host RAM used, how unusual the peak is	**0.9 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host RAM used, P(≥50%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy

2. Per-container

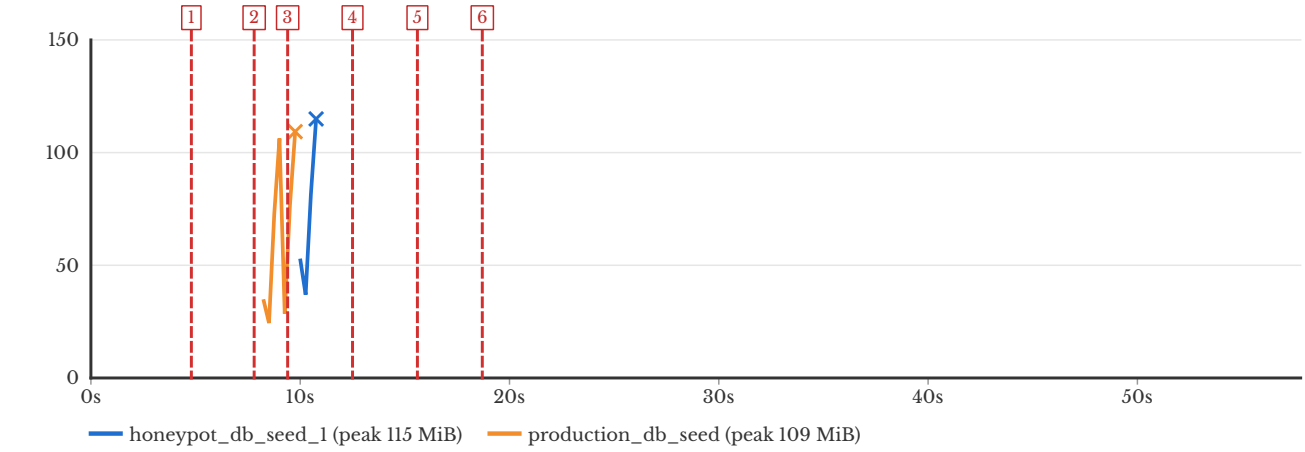
Container CPU is in % of one core: 100% is one core fully busy, so a multi-threaded container goes above 100%. This host has 20 cores, so the ceiling is 2000%. Grouped by role, one CPU and one memory chart per group, so every line in a chart has a colour of its own; when one or two containers dwarf the rest, they get a chart of their own and the others are drawn on their own scale.

One-shot jobs

One-shot jobs: container CPU %

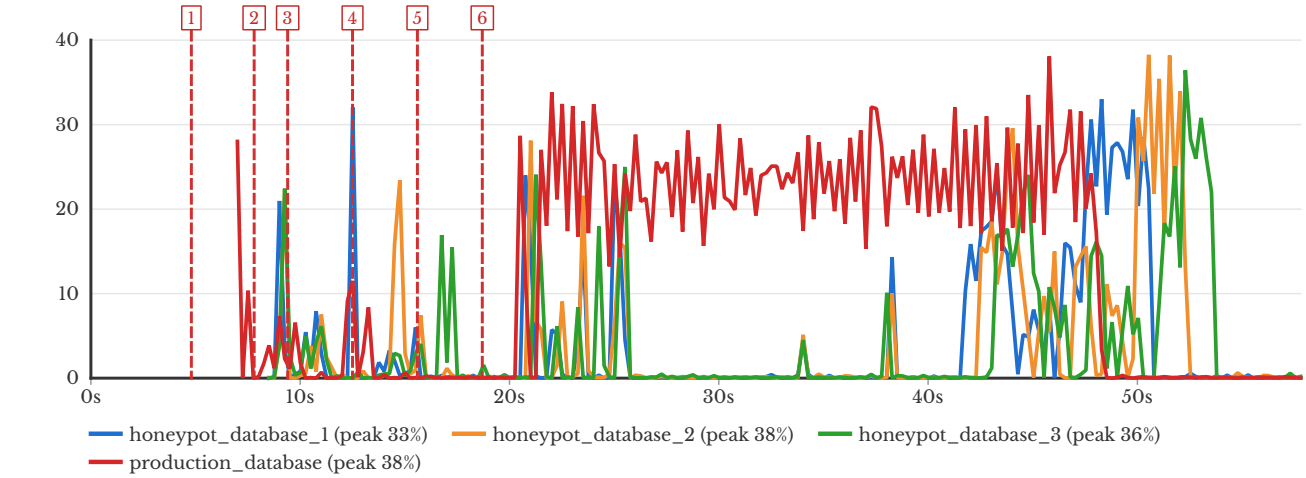


One-shot jobs: container memory (MiB)

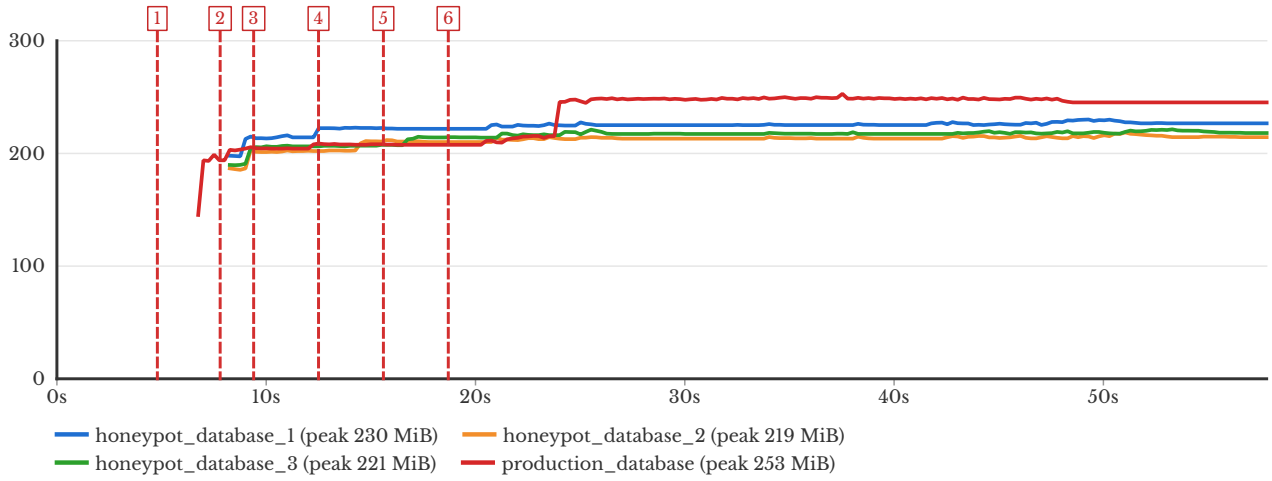


Databases

Databases: container CPU %

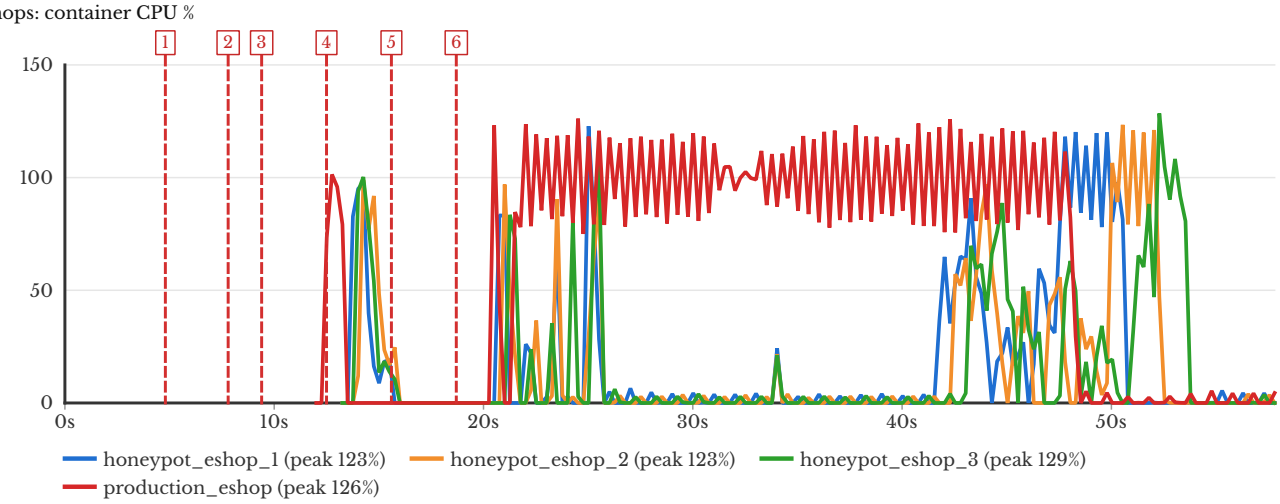


Databases: container memory (MiB)

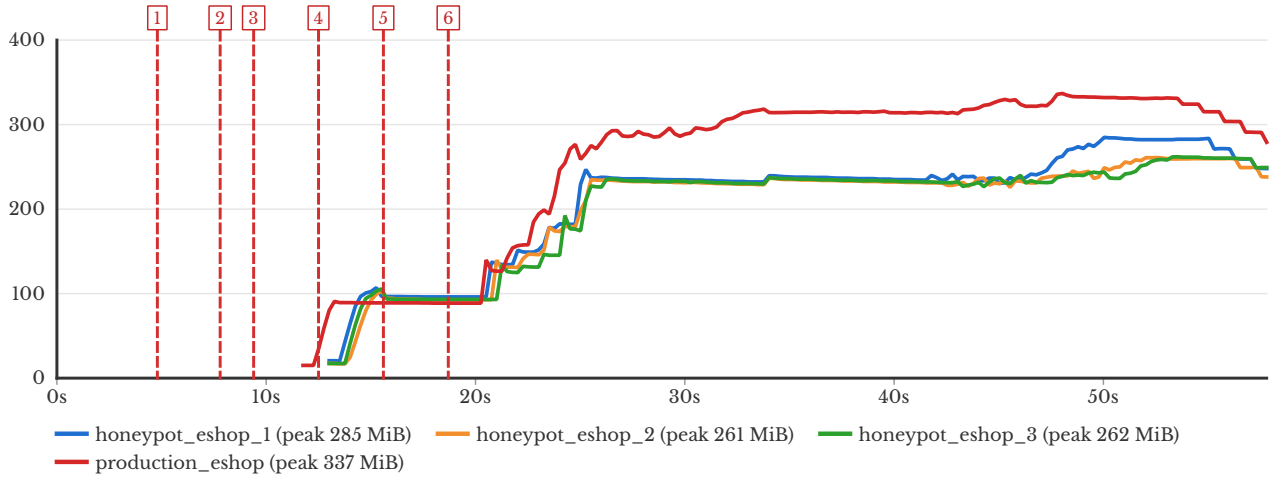


Shops

Shops: container CPU %

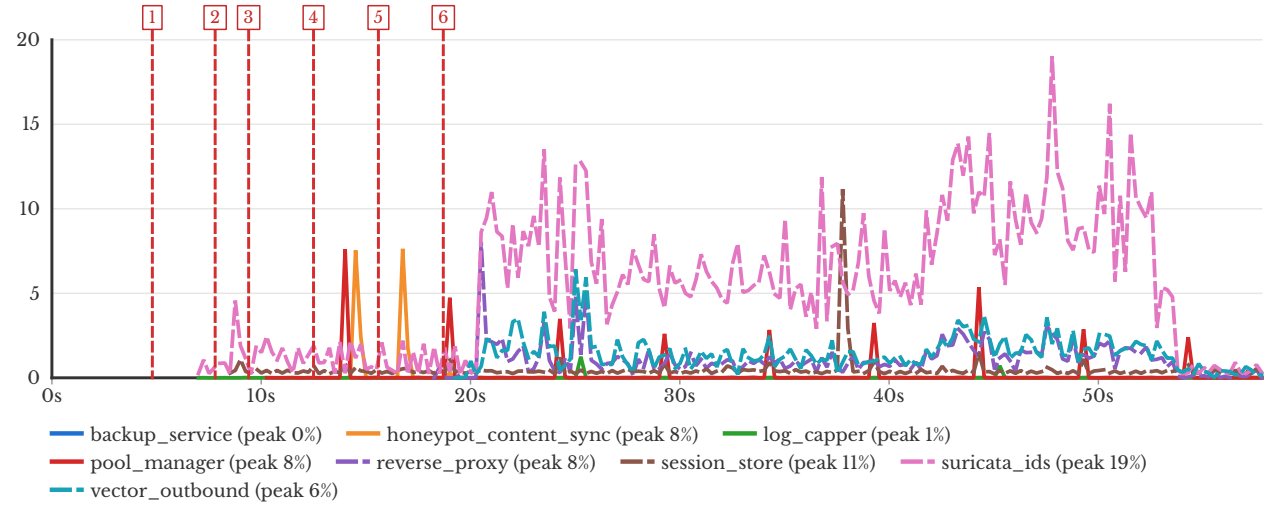


Shops: container memory (MiB)

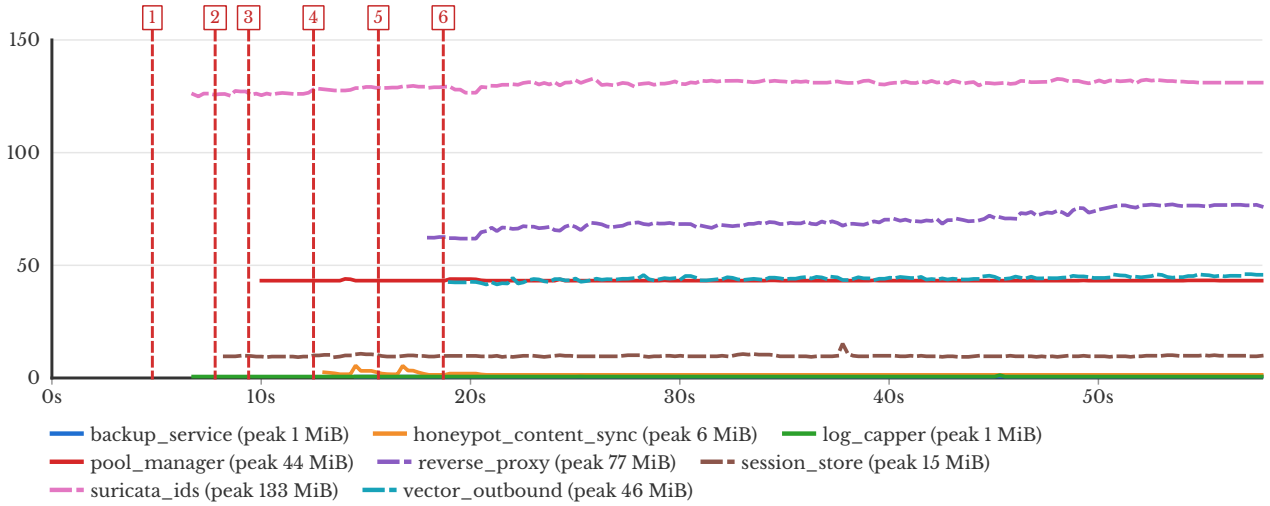


Edge & monitoring

Edge & monitoring: container CPU %



Edge & monitoring: container memory (MiB)



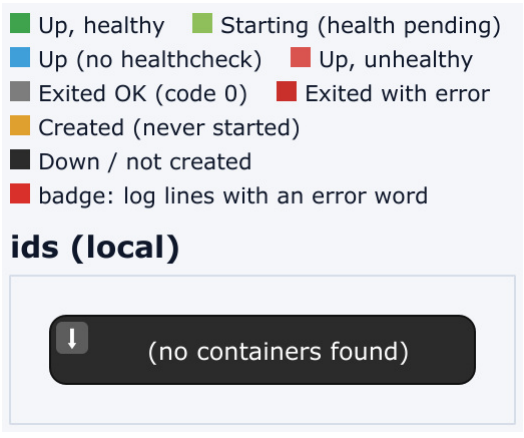
Probability metrics: per container						
Container	**Samples / over**	**CPU mean ± SD**	**CPU p95 / peak**	**P(CPU ≥ 50%)**	**Mem MiB mean ± SD**	**Mem MiB peak**
backup_service	174 / 43.3 s	0.0% ± 0.0	0.0% / 0.0%	0.0%	1 ± 0	1
honeypot_content_sync	180 / 44.8 s	0.1% ± 0.8	0.0% / 7.6%	0.0%	2 ± 1	6
honeypot_database_1	199 / 49.5 s	4.1% ± 8.1	23.6% / 33.0%	0.0%	224 ± 5	230
honeypot_database_2	199 / 49.5 s	4.0% ± 8.1	22.5% / 38.3%	0.0%	212 ± 5	219
honeypot_database_3	199 / 49.5 s	3.8% ± 7.4	22.1% / 36.4%	0.0%	215 ± 6	221
honeypot_db_seed_1*	4 / 0.8 s	86.5% ± 10.4	96.4% / 97.8%	100.0%	71 ± 34	115
honeypot_eshop_1	180 / 44.8 s	17.6% ± 32.4	91.3% / 122.8%	16.2%	212 ± 66	285
honeypot_eshop_2	180 / 44.8 s	16.3% ± 30.7	90.8% / 123.4%	13.4%	202 ± 64	261
honeypot_eshop_3	180 / 44.8 s	15.8% ± 29.7	88.4% / 128.5%	15.1%	201 ± 64	262
log_capper	205 / 51.0 s	0.0% ± 0.1	0.0% / 1.3%	0.0%	1 ± 0	1
pool_manager	192 / 47.8 s	0.2% ± 0.9	0.0% / 7.6%	0.0%	43 ± 0	44
production_database	205 / 51.0 s	13.2% ± 12.2	31.5% / 38.1%	0.0%	234 ± 20	253
production_db_seed*	7 / 1.5 s	95.4% ± 4.1	99.0% / 99.3%	100.0%	65 ± 36	109
production_eshop	185 / 46.0 s	61.2% ± 50.1	120.8% / 126.2%	60.9%	256 ± 96	337
reverse_proxy	160 / 39.8 s	1.1% ± 1.0	2.6% / 8.0%	0.0%	70 ± 4	77
session_store	199 / 49.5 s	0.5% ± 0.8	0.8% / 11.2%	0.0%	10 ± 0	15
suricata_ids	205 / 51.0 s	5.4% ± 4.1	12.2% / 19.0%	0.0%	130 ± 2	133
vector_outbound	156 / 38.8 s	1.5% ± 1.0	3.3% / 6.4%	0.0%	44 ± 1	46

Each row covers only the samples taken while that container was running (count and the time between its first and last one). P(CPU ≥ 50%) is the share of those samples at half a core or more, i.e. the chance of catching it that busy at a random moment of its run. A peak far above the p95 means a short burst (typically start-up work or a first healthcheck) rather than sustained load. * fewer than 10 samples (the container ran only briefly while recorded): its figures describe a moment, not the whole run.

3. Global Visualization page as the stack changes state

Only this target's tiles, drawn from the same *docker compose ps* result the report logged at that moment, so tile colours and the event log agree. Badges are explained under Warnings.

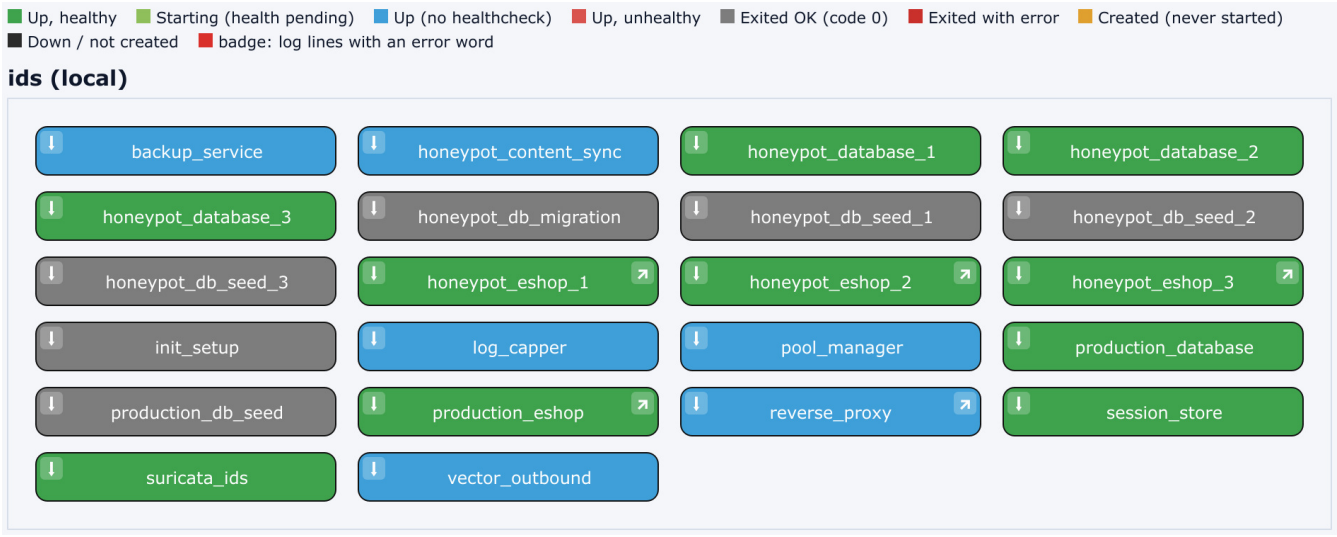
Before: stack idle / not yet changed (before t = 0)



Starting: containers coming up (at 4.8 s)



All containers healthy (at 19.2 s)



Appendix: event log

The events behind the event key, one per row.

Marker	Elapsed	Event
1	4.8 s	backup_service created
1	4.8 s	honeypot_content_sync created
1	4.8 s	honeypot_database_1 created
1	4.8 s	honeypot_database_2 created
1	4.8 s	honeypot_database_3 created
1	4.8 s	honeypot_db_migration created
1	4.8 s	honeypot_db_seed_1 created
1	4.8 s	honeypot_db_seed_2 created
1	4.8 s	honeypot_db_seed_3 created
1	4.8 s	honeypot_eshop_1 created
1	4.8 s	honeypot_eshop_2 created
1	4.8 s	honeypot_eshop_3 created

Marker	Elapsed	Event
1	4.8 s	init_setup created
1	4.8 s	pool_manager created
1	4.8 s	production_database created
1	4.8 s	production_db_seed created
1	4.8 s	production_eshop created
1	4.8 s	reverse_proxy created
1	4.8 s	session_store created
1	4.8 s	suricata_ids created
1	4.8 s	vector_outbound created
1	4.8 s	log_capper created
1	6.3 s	init_setup finished
1	6.3 s	production_database running, health pending
1	6.3 s	suricata_ids running, health pending
1	6.3 s	log_capper started
2	7.8 s	honeypot_database_1 healthy
2	7.8 s	honeypot_database_2 running, health pending
2	7.8 s	honeypot_database_3 healthy
2	7.8 s	production_database healthy
2	7.8 s	production_db_seed started
2	7.8 s	session_store healthy
3	9.4 s	honeypot_database_2 healthy
3	9.4 s	honeypot_db_migration finished
3	9.4 s	honeypot_db_seed_1 started
3	9.4 s	pool_manager started
3	9.4 s	suricata_ids healthy
3	10.9 s	honeypot_db_seed_2 started
3	10.9 s	honeypot_db_seed_3 started
3	10.9 s	production_db_seed finished
3	10.9 s	production_eshop running, health pending
4	12.5 s	honeypot_content_sync started
4	12.5 s	honeypot_db_seed_1 finished
4	12.5 s	honeypot_db_seed_2 finished
4	12.5 s	honeypot_db_seed_3 finished
4	12.5 s	honeypot_eshop_1 running, health pending
4	12.5 s	honeypot_eshop_2 running, health pending
4	12.5 s	honeypot_eshop_3 running, health pending
4	14.0 s	backup_service started
4	14.0 s	production_eshop healthy
5	15.6 s	honeypot_eshop_1 healthy
5	15.6 s	honeypot_eshop_2 healthy
5	15.6 s	honeypot_eshop_3 healthy
5	17.1 s	reverse_proxy started
6	18.7 s	vector_outbound started